

Introduction aux promesses

Les promesses sont un concept fondamental en JavaScript pour gérer les opérations asynchrones. Elles permettent de représenter une valeur qui peut être disponible maintenant, dans le futur, ou jamais. Lors des appels au serveur, les promesses sont souvent utilisées pour gérer les réponses asynchrones.

Vous pouvez créer vos propres promesses en plus des fonctions qui en retournent déjà, comme `fetch()`.

Création d'une promesse

Une promesse peut être créée en utilisant le constructeur `Promise` et `new`. Le constructeur prend une fonction en argument qui reçoit deux paramètres : `resolve` et `reject`.

`resolve` est appelé lorsque l'opération asynchrone réussit, tandis que `reject` est appelé en cas d'échec.

Vous pouvez ensuite utiliser les méthodes `.then()` et `.catch()` pour gérer les résultats ou les erreurs de la promesse.

```
const maPromesse = new Promise(function (resolve, reject) {
  // Simuler une opération asynchrone
  setTimeout(() => {
    const succes = true; // Changer cette valeur pour tester le succès ou
    l'échec
    if (succes) {
      resolve("Opération réussie !");
    } else {
      reject("Opération échouée.");
    }
  }, 2000);
});

maPromesse
  .then((message) => {
    console.log(message); // "Opération réussie !" si la promesse est résolue
  })
  .catch((erreur) => {
    console.error(erreur); // "Opération échouée." si la promesse est rejetée
  });
```

async/await

L'`async/await` est une syntaxe introduite dans ES2017 qui permet de travailler avec des promesses de manière plus synchrone. Une fonction déclarée avec le mot-clé `async` retourne toujours une promesse. Le mot-clé `await` peut être utilisé à l'intérieur d'une fonction `async` pour attendre la résolution d'une promesse avant de continuer l'exécution du code.

On utilise cette syntaxe pour rendre le code plus lisible et éviter les chaînes de `.then()`.

La fonction doit être marquée avec le mot-clé `async` pour pouvoir utiliser `await` à l'intérieur.

```

async function maFonction() {
  try {
    const resultat = await uneOperationAsynchrone(); // Exemple un fetch
    console.log(resultat);
  } catch (erreur) {
    console.error(erreur);
  }
}

```

Utilisation avec fetch

L'API `fetch` est une interface moderne pour effectuer des requêtes HTTP. Elle retourne une promesse qui se résout avec l'objet `Response` lorsque la requête est terminée.

Version avec "then/catch" :

```

fetch("/api/recupererDonnees.php")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error("Erreur :", error);
  });

```

Version avec "async/await" :

```

async function recupererDonnees() {
  const response = await fetch("/api/recupererDonnees.php");
  const data = await response.json();
}

```

Try/Catch

Le bloc `try/catch` est utilisé pour gérer les erreurs dans le code JavaScript. Lorsqu'une erreur se produit dans le bloc `try`, l'exécution est immédiatement transférée au bloc `catch`, où l'erreur peut être gérée. Comme avec le `async/await`, il n'y a pas de bloc `catch` par défaut avec les promesses, donc on utilise `try/catch` pour capturer les erreurs.

```

async function recupererDonnees() {
  try {
    const response = await fetch("/api/recupererDonnees.php");
    const data = await response.json();
  } catch (error) {
    console.error("Erreur :", error);
  }
}

```

}

}